

人工智能赋能学术研究 科研生产力提升的路径、方法与实践

报告人：李嘉晟

2026 年 5 月 27 日

讲座目标

- 目标是从零开始建立可操作的使用框架。
- 重点不是穷尽功能，而是帮助大家理解：何时适合使用、怎样建立情景、怎样组织工作流、何时必须人工接管。
- 现场演示将以一个项目模板和一个主案例为中心，尽量看到完整的最小闭环。

内容安排

从零开始认识 Codex

基本使用方法

主案例：CFPS、Stata 与双重差分

扩展方向

结语

Codex、Claude Code 与豆包的区别和联系

- 三者都属于大语言模型驱动的人工智能工具，都可以理解自然语言并生成文本。
- 不同之处在于，Codex 与 Claude Code 更强调 **项目环境中的执行能力**，即读取文件、修改文件、运行命令、调用工具。
- 豆包更接近通用对话式助手，适合问答、总结和日常内容生成；而 Codex 更适合围绕一个具体项目持续工作。
- 因而如果目标是做科研工作流，重点往往不只是“会不会回答”，而是“能不能进入项目并持续执行”。

实际上Codex、Claude Code、Antigravity这些软件的差别不大。不过就演示的便携性考虑，本次讲座使用Codex进行演示。

每次对话时人工智能在做什么

- 从模型视角看，每次对话本质上都是在读取当前上下文，并预测接下来最合适输出的文本。
- 所谓“上下文”不仅包括当前提问，还包括历史对话、项目文件、规则说明、工具返回结果与系统约束。
- 当上下文很长、任务很复杂时，模型的表现很大程度取决于 **情景是否建立清楚**，而不只是提问句子是否漂亮。
- 这也是为什么项目式使用人工智能时，经常需要先让它读文件、读目录、读规则，而不是立刻要求它执行。

为什么 Codex 这一类的AIAPP更适合科研项目

- 它不仅能回答问题，还能在项目中主动读取结构、修改文件、运行脚本、编译文档并调用可用软件。
- 这使其更像一个 **可执行的研究助理**，而不只是一个对话框。
- 对科研而言，很多高耗时工作恰恰发生在文件整理、路径配置、脚本补全、日志记录与结果输出这些执行性环节。
- 因而 Codex 的优势，在于把“会说”推进到了“会在项目环境中做事”。

当前 Codex 好用的几个原因

- 任务拆分能力：能够把复杂要求拆成若干步骤，更适合构成工作流。
- 文件读写能力：可以在项目内部持续修改脚本、文稿与规则文件。
- 情景细化能力：可以通过目录、说明文档、日志与样例快速建立任务背景。
- 接管应用能力：在允许的条件下可以调用 Stata、Python、LaTeX 等环境执行任务。

应当如何理解它的边界

- 人工智能可以显著提升执行效率，但并不自动等于研究判断正确。
- 它适合承担清洗、整理、草拟、检查、补全与格式统一等任务。
- 它不适合替代研究者完成研究问题选择、识别策略判断、结构设定取舍与最终学术表态。
- 一个稳妥的原则是：把 **Codex** 当研究助理，而不是当研究负责人。

为什么要先有一个项目模板

- 刚接触时最大的困难往往不是不会提问，而是不知道如何给人工智能建立稳定情景。
- 项目模板的作用，是把目录结构、规则、脚本位置、数据说明和输出位置预先安排好。
- 这样一来，Codex 进入项目后就可以快速理解“这里是什么、能做什么、结果要放到哪里”。
- 因而项目模板本身，就是工作流的一部分。

一个入门模板至少应包含什么

- README.md: 项目目标、研究问题与基本说明。
- data/ 或数据说明文档: 样本来源、变量口径与路径约定。
- scripts/: Stata、Python 或其他脚本目录。
- output/: 结果表格、日志与文稿输出位置。
- WORKFLOW.md: 任务规则、不可擅自更改的边界与验证要求。

开始使用时应先做什么

- 第一，不要立刻说“帮我做研究”，而应先要求 Codex 阅读目录结构与关键文件。
- 第二，让它总结这个项目当前在做什么、已有材料是什么、还缺什么。
- 第三，再让它说明可用工具与可执行路径，例如 Stata、Python、LaTeX 是否已经配置。
- 这一过程的目标，是先建立 **工作情景**，再进入正式执行。

最重要的方法：要么给计划，要么先让它出计划

- 如果研究者已经有成熟方案，应尽量把步骤、约束、输出格式与不可触碰边界说清楚。
- 如果方案还不完整，则不宜直接放手执行，而应先要求模型给出计划，再由研究者修改和确认。
- 这一步对于工作流质量影响极大，因为抽象任务如果缺乏计划，模型很容易偏离研究者真实意图。
- 因而一个非常实用的原则是：先对齐理解，再启动执行。

一个可直接使用的提问模板

建议模板

这是我的任务；这是当前项目背景；这是我希望的输出；这是不能做的事。请先给我一个计划，不要直接修改文件。

- 在计划确认后，再继续要求执行，并明确每一步完成后要报告什么。
- 如果任务较长，应要求它保留日志、说明修改原因，并在发现异常时先停下。
- 对初学者而言，这一模板比追求复杂提示词更重要。

主案例定位

CFPS、Stata 与双重差分

用一个主案例讲清项目模板、工作流、环境配置与文稿输出

假设研究情景

- 研究问题设定为：中美贸易摩擦是否影响中国劳动力市场。
- 以沿海城市作为实验组，非沿海城市作为对照组，冲击发生时间设定为 2018 年。
- 使用 CFPS 数据进行双重差分分析，并最终输出表格和一份简短的 LaTeX 文稿。
- 这一案例的重点不在于结论本身，而在于 **展示一个最基本科研工作流如何被组织起来。**

第一步：配置环境

- 首先需要向 Codex 明确 Stata 的路径、工作目录、数据位置、脚本目录与输出目录。
- 其次需要告诉它当前样本的基本定义、变量口径、日志存放位置与表格格式要求。
- 对初学者而言，这一步看似技术性很强，实则是在建立“人工智能可以在什么范围内行动”的边界。
- 环境不清楚时，后续 workflow 再好，也会频繁被打断。

Stata、Python 与 LaTeX 的路径一般在哪里

- **Stata**: 在 Windows 中通常位于 `C:\Program Files\StataXX\` 下, 例如 `StataMP-64.exe` 或 `StataSE-64.exe`。
- **Python**: 可能来自 Anaconda、系统安装版或虚拟环境, 常见位置包括 `C:\Users\用户名\anaconda3\python.exe`、`C:\Python311\python.exe`, 或项目虚拟环境中的 `.venv\Scripts\python.exe`。
- **LaTeX**: 如果使用 TeX Live, 通常位于 `C:\texlive\年份\bin\win32\`, 例如 `xelatex.exe` 或 `latexmk.exe`。
- 如果不确定路径, 可以先在终端中用 `Get-Command python`、`Get-Command xelatex`, 或直接在软件快捷方式属性中查看其真实位置。

第二步：让它先读项目，再提计划

- 在正式清洗和回归之前，应先要求 Codex 阅读项目文件结构、变量说明与脚本框架。
- 然后要求它提出一个可执行计划，例如样本整理、变量构造、描述统计、基准回归与输出文稿。
- 此时研究者要检查的重点，是是否遗漏关键步骤、是否误解了变量口径、是否默认进行了不合理删值。

第三步：清理数据并形成可复用脚本

- 接下来可以让 Codex 根据计划生成或补全 do-file，完成变量整理、缺失检查与样本筛选。
- 在这一步中，人工智能的优势主要体现在重复劳动的自动化，而不是研究判断的替代。
- 研究者需要重点关注：变量定义是否被正确理解、处理组与对照组是否构造合理、时间变量是否准确。
- 如果这一阶段做得规范，后续估计和写作都会稳定得多。

第四步：执行双重差分估计

- 在样本和变量确认后，可以让 Codex 组织双重差分脚本、固定效应、聚类标准误与结果输出格式。
- 若需要扩展到异质性分析或稳健性检验，也应作为显式任务提出，而不是临时追加一句口头指令。
- 关键不在于模型会不会写 `reg`，而在于能否 按研究规则稳定地写出可检查的 `reg`。

第五步：结果解释与人工检查点

- 结果出来以后，可以让 Codex 先生成描述性解释，但必须要求区分结果、含义与边界。
- 研究者要重点检查：识别逻辑是否被过度延伸、统计显著是否被误写成经济显著、文稿语气是否过强。
- 因而人工检查点应放在结论解释之前，而不是等到整篇文稿写完之后再回头修正。

第六步：生成简短文稿

- 可以进一步要求 Codex 将主要表格和结果说明写入一个简短的 LaTeX 文档。
- 这个文档不必一开始就追求完整论文结构，但应至少包含研究背景、设定说明、主要结果与简短讨论。
- 这样做的好处在于，听众可以清楚看到：一个工作流不仅产生代码，也产生 **可汇报、可写作、可继续扩展的成果**。

这个案例想传达的工作流观念

1. 先建立项目情景，再开始执行。
2. 要么给出具体计划，要么先让人工智能提出计划并人工修正。
3. 在清洗、估计、解释与写作之间设置检查点。
4. 把结果沉淀成脚本、日志与文稿，而不是停留在一次性对话中。

从简化式到结构估计

- 在更复杂的研究中，Codex 也可以协助 Python 环境下的结构估计工作。
- 例如在实证产业组织中，可以借助 pyblp 组织工具变量、结构估计与简单反事实模拟。

Bonus: 如何调用浏览器

- 在一些任务中，研究材料并不在本地文件，而是位于网页、政策页面、数据平台或在线说明文档中。
- 如果允许浏览器协同，Codex 就可以辅助读取页面文本、核对公开信息与收集网页证据。
- 这一能力适合作为公开信息搜集与页面核验的补充，而不宜替代研究者的解释与判断。
- 因而浏览器接入最适合讲成一个 bonus: 它扩展的是取材能力，而不是免验证的权力。

一小时半之后最希望大家记住什么

1. Codex等AI软件的优势不只在于会回答问题更在于能进入项目环境持续工作。
2. 高质量使用的关键不是更华丽的提示词，而是 **更清楚的情景、更明确的计划和更完整的工作流。**
3. 对科研而言，最现实的目标不是让人工智能替我们做研究，而是让它帮助我们更高效地组织研究。

谢谢大家

谢谢大家

欢迎交流